ÉTS
Le génie pour l'industrie
Département de génie logiciel et des TI

UQÀM | Département d'informatique

La gestion plus moderne des projets logiciels (suite)

Professeur Alain April, ing. PHD
(avec la contribution de Sylvie Trudel, PhD, UQAM
Complété par Bruel Gérançon, PhD, UQAM



Le contenu de ce document est protégé par le droit d'auteur. Ce document peut être utilisé à des fins d'étude personnelle seulement. Aucune partie de cette œuvre ne peut être transmise ou reproduite d'une quelconque façon sans la permission du détenteur du droit d'auteur.



Les méthodes agiles (suite)

Méthodes agiles (suite)

- Scrum
- XP (eXtreme Programming)
- Agile UP
- Kanban
- Agile à grande échelle
- DevOps
- Technique du Sunset Graph
- TP2 - Énoncé de planification initiale



Défi : développer rapidement une application permettant aux étudiants
de consulter le menu, commander et éviter la file d'attente.

Introduction à Scrum

Cas pratique : une application de commande pour la cafétéria

- Imaginez que l'UQAM veut développer une petite application Web permettant aux étudiants de commander leur repas avant d'arriver à la cafétéria.
- Client: Vous avez 4 mois. Revenez me voir lorsque toute l'application sera terminée. »
- **Pendant 4 mois, l'équipe analyse, conçoit et programme tout le système. Au bout de 4 mois, elle présente enfin l'application au responsable de la cafétéria.**
- **Client :**
- « C'est intéressant... mais entre-temps, nos besoins ont changé. Nous voulons maintenant permettre aux étudiants de choisir une heure de récupération, certains produits sont épuisés pendant la journée et nous avons aussi besoin d'un système permettant d'annuler une commande. »
- Quel est le problème ici? L'équipe a-t-elle nécessairement mal travaillé? »

Priorité	Besoin
1	Consulter le menu
2	Voir le prix des repas
3	Ajouter un repas au panier
4	Passer une commande
5	Choisir l'heure de récupération
6	Payer en ligne
7	Annuler une commande
8	Recevoir une confirmation

Situation 1: Une longue attente avant le feedback

Besoins du client

4 mois de développement

Produit terminé

Le client : « Nos besoins ont changé : heure de récupération, produits épuisés, annulation des commandes… »

Question aux étudiants

Quel est le problème dans cette façon de gérer le projet ?

- Feedback trop tardif
- Besoins qui changent
- Risque de mauvais produit

Situation 2: Travailler avec Scrum

Product Backlog

Sprint 1
Menu + prix

Feedback client

Sprint 2
Panier

Feedback client

Chaque Sprint produit un résultat utilisable
et permet d'apprendre.

Exemple de progression par Sprints

Sprint 1	Consulter le menu et les prix
Sprint 2	Ajouter au panier et commander
Sprint 3	Choisir l'heure de récupération
Sprint 4	Payer et recevoir une confirmation

Alors, qu’est-ce que Scrum ?

Développer progressivement → montrer → obtenir du feedback → s’adapter

Product Owner

Scrum Master

Developers

Sprint

Question : Pourquoi le feedback fréquent réduit-il le risque du projet ?

Méthodes agiles

Scrum

- Méthode plus adoptée pour la gestion de projet et de la gestion des exigences
- Son objectif est d'augmenter la qualité de la communication et de développer des solutions:

– En encourageant les équipes auto gérées et multidisciplinaires

– En rapprochant le PO de l'Équipe de développement

– En planifiant la réalisation des exigences (items du product backlog) par ordre de valeur d'affaires

XP (eXtreme Programming)

- Méthode la plus technique
- Pratiques rigoureuses d'ingénierie logicielle augmentant la qualité technique et la souplesse du code développé
  - Développement piloté par les tests (TDD)
  - Intégration continue
  - Refactoring
  - Collaboration entre les membres de l'équipe de collaboration (pair programming)
  - NB: XP = une méthode agile centrée sur les pratiques techniques qui permettent de produire un code de qualité et facilement adaptable aux changements.

- Pourquoi les pratiques de XP augmentant la qualité technique et la souplesse du code développé?
- Ces pratiques augmentent :
  - la qualité technique : les tests détectent rapidement les erreurs, le travail en binôme favorise la révision du code et le refactoring maintient un code propre;
  - la souplesse du code : un code simple, testé et régulièrement amélioré est généralement plus facile à modifier lorsque les exigences changent.

Agile UP

- Une adaptation de la méthode RUP (*Rational Unified Process)*
- 4 grandes périodes itératives et incrémentales dans le cycle de développement
  - Évaluation (inception)
  - Élaboration
  - Construction
  - Transition

Kanban

- Méthode agile axée sur la visualisation et l'optimisation du flux de travail
- Méthode inspirée des méthodes manufacturières de compagnies tels que Toyota
- Applique l'amélioration pour augmenter l'efficacité des processus de développement
- L'élimination du gaspillage
- En livrant peu, mais fréquemment de manière à réduire l'inventaire du développement

Kanban

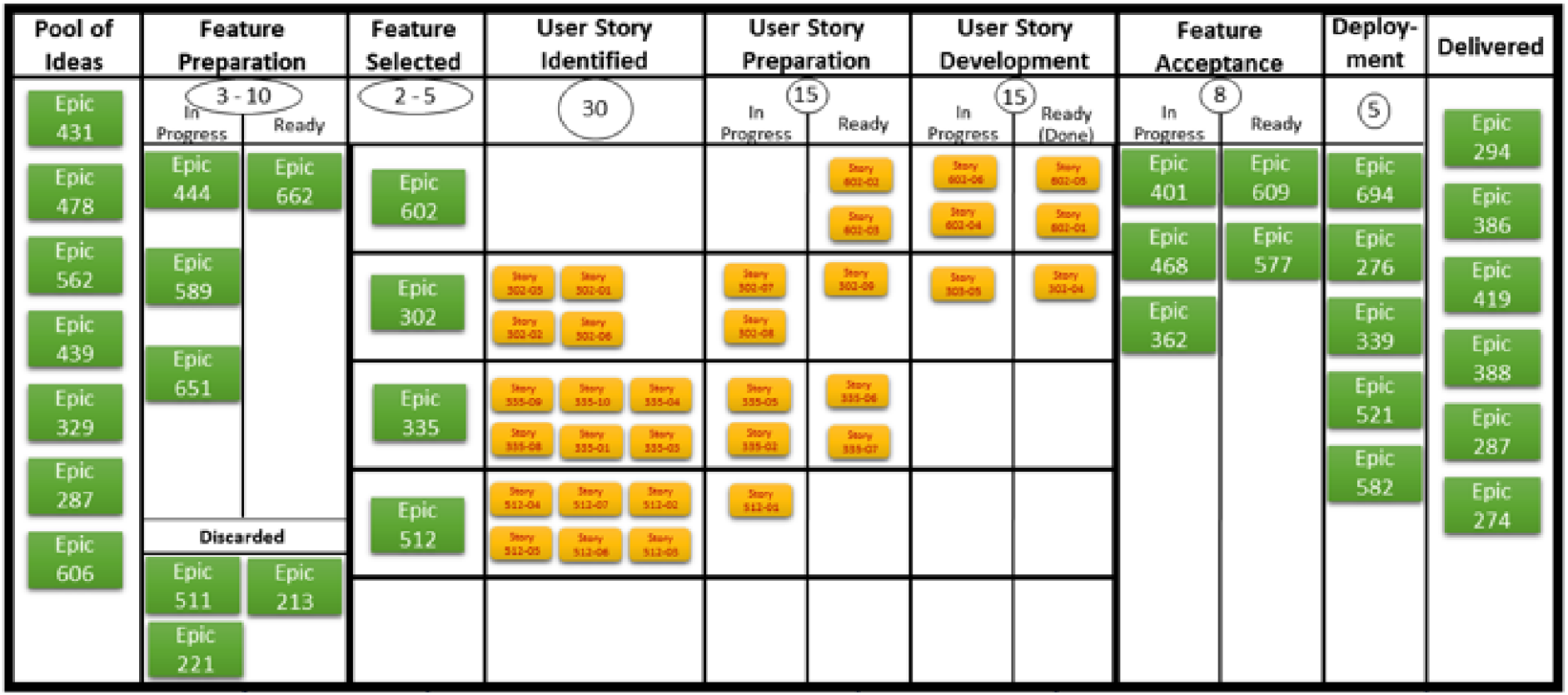
Kanban

Éléments clés de Kanban	Définition
Tableau Kanban	Tableau visuel représente les différentes étapes du processus de développement : À faire, En cours, En revue, Terminé, etc.).
Cartes	Chaque tâche est représentée par une carte contenant des informations essentielles (description, priorités, responsables). Les cartes se déplacent à travers les colonnes du tableau au fur et à mesure de l'avancement du projet
Feedback continu	Des réunions régulières à travers des revues de sprint
Amélioration continue	On se focalise sur l'analyse des performances et l'adaptation des pratiques pour accroître l'efficacité et la qualité des livrables.

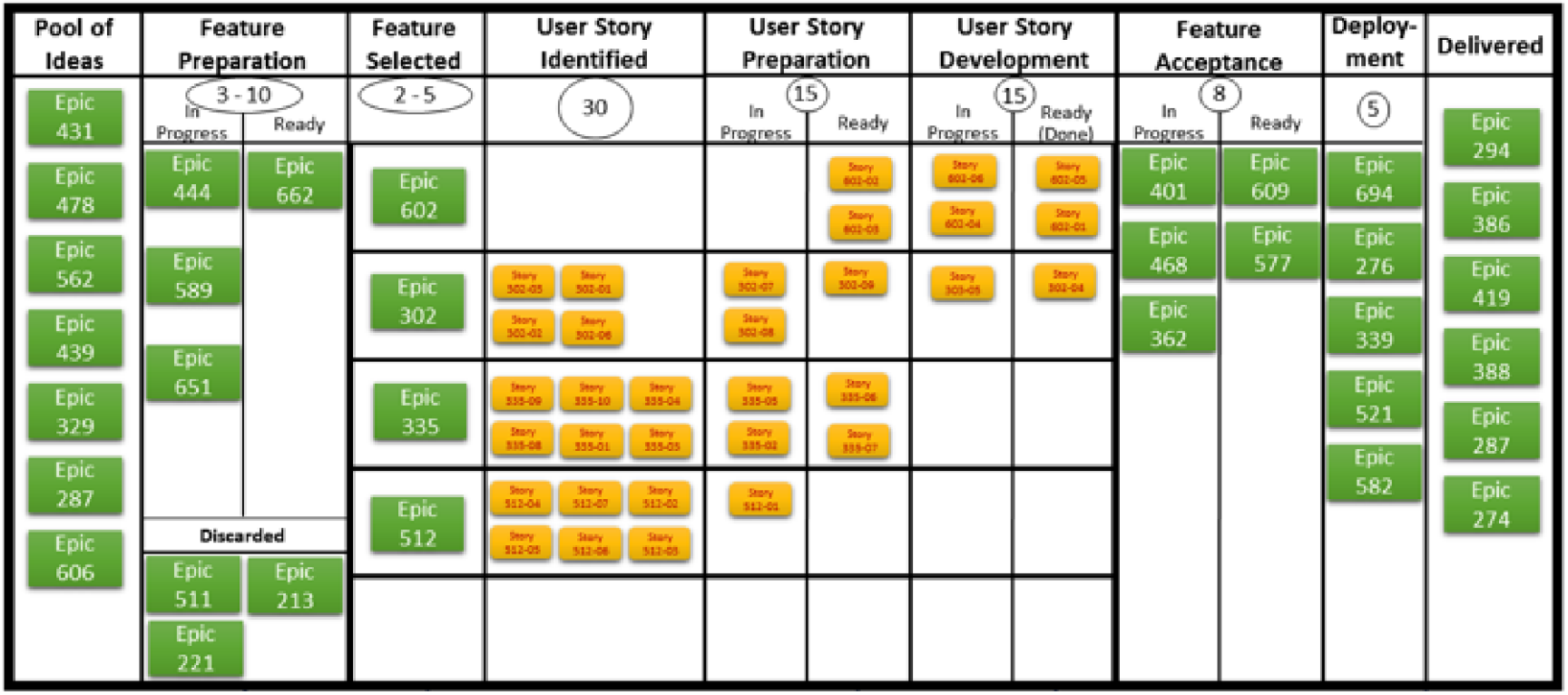
3.4 - Kanban

- Kanban:
  - Le découpage des tâches d'une manière égale (ex. 2 à 3 jours);
  - Une limite de tâches par colonne
  - Des sous-colonnes avec la notion de « À faire, en cours » et « terminé »
  - Des règles pour dire qu'une tâche est vraiment terminée;
  - Pas d'échéance d'itérations, pas de réunions de planification, etc.
- Kanban = Itératif
- On mesure le « temps de cycle », soit la durée entre le moment où on commence à travailler sur une story et le moment où elle est « terminée » → On cherche à l'optimiser

3.4 - Kanban


Pool of Ideas
Epic 431
Epic 478
Epic 562
Epic 439
Epic 329
Epic 287
Epic 606
Feature Preparation
3 - 10
In Progress
Ready
Epic 444
Epic 589
Epic 651
Epic 662
Discarded
Epic 511
Epic 213
Epic 221
Feature Selected
2 - 5
Epic 602
Epic 302
Epic 335
Epic 512
User Story Identified
30
User Story Preparation
15
In Progress
Ready
User Story Development
15
In Progress
Ready (Done)
Feature Acceptance
8
In Progress
Ready
Epic 401
Epic 468
Epic 362
Epic 609
Epic 577
Deploy-ment
5
Epic 694
Epic 276
Epic 339
Epic 521
Epic 582
Delivered
Epic 294
Epic 386
Epic 419
Epic 388
Epic 287
Epic 274

3.4 - Kanban


Pool of Ideas
Feature Preparation
3 - 10
In Progress
Ready
Feature Selected
2 - 5
User Story Identified
30
User Story Preparation
15
In Progress
Ready
User Story Development
15
In Progress
Ready (Done)
Feature Acceptance
8
In Progress
Ready
Deploy-ment
5
Delivered
Epic 431
Epic 478
Epic 562
Epic 439
Epic 329
Epic 287
Epic 606
Epic 444
Epic 589
Epic 651
Epic 662
Discarded
Epic 511
Epic 213
Epic 221
Epic 602
Epic 302
Epic 335
Epic 512
Epic 401
Epic 468
Epic 362
Epic 609
Epic 577
Epic 694
Epic 276
Epic 339
Epic 521
Epic 582
Epic 294
Epic 386
Epic 419
Epic 388
Epic 287
Epic 274

Epic et User Stories – Exemple

EPIC : Gérer les commandes de la cafétéria

→ User Story 1 : Consulter le menu

→ User Story 2 : Ajouter un repas au panier

→ User Story 3 : Passer une commande

→ User Story 4 : Payer en ligne

→ User Story 5 : Annuler une commande

Epic = grand besoin → plusieurs User Stories → réalisées progressivement.

3.4 - Kanban

- Kanban:
  - Le découpage des tâches d'une manière égale (2 à 3 jours): Cette approche évite les réunions de planification et l'estimation, car tout est brisé en petits morceaux
  - Apprenez à découper en tâches de 2-3 jours:
    - Par élément fonctionnel (exemple une recherche par nombreux paramètres)
    - Par fonctionnalité CRUD
    - Par les étapes ou les rôles d'une transaction (enregistrement, logon, ..)
    - Par cas de test
    - Par difficulté d'implantation

Agile à grande échelle

C’est quoi l’agile à grande échelle?

Agile à grande échelle

- Appliquer des principes et pratiques Agile au sein d'organisations de grande taille, impliquant plusieurs équipes travaillant ensemble sur des projets complexes.
- Comment maintenir les avantages de l'Agilité tout en gérant les défis liés à la coordination, la communication, et l'alignement entre plusieurs équipes?
    - Flexibilité, livraison rapide, amélioration continue

3.5 – Agile à grande échelle

- Il y a plusieurs méthodes pour coordonner le travail d'un ensemble d'équipes Agiles ensemble:
  - Les stand-up meetings diffèrent un peu:
    - Instaurer la livraison continue de toutes les équipes
    - Créer un Backlog commun à toutes les équipes
    - Favoriser une culture collaborative
    - Considérer l'utilisation d'une méthode à grande échelle:

Agile à grande échelle

- Cadre de gestion Agile à grande échelle qui permet de coordonner plusieurs équipes travaillant sur un même produit complexe
  - Scaled Agile Framework de Dean Leffingwell (SAFe)
  - Il combine les principes Agile et DevOps pour favoriser l'alignement, la collaboration, et la livraison rapide de valeur.
  - Disciplined Agile Delivery de Scott Ambler (DAD)
  - Large Scale Scrum de Craig Larman (LeSS)
  - Scrum of Scrums (SoS)
  - LeadingAgile

Agile à grande échelle

Le Scrum of Scrums

- Réunion de coordination utilisée dans le cadre de l'Agile à grande échelle pour synchroniser plusieurs équipes Scrum travaillant sur un projet commun.
- Représentants de chaque équipe se réunissent régulièrement (chaque jour ou chaque semaine)
  - Partage de l'avancement des équipes.
  - Identification et résolution des interdépendances entre les équipes.
  - Traitement des obstacles affectant les équipes.

Cas pratique

- Comment utiliser l’agile à grande dans le cadre d’un projet de développement des Logiciels embarqués pour l'automobile où l’équipe de développement comporte 50 développeurs?

Cas pratique (suite)

- Comment peut-on assurer l'intégration continue dans un tel contexte?
- Autrement dit comment tester et déployer rapidement les fonctionnalités?
  - Les différentes équipes développent leurs fonctionnalités séparément, mais leur code doit être intégré fréquemment dans une version commune du système.
  - On met donc en place un pipeline CI/CD automatisé :

DevOps

- Quel est le contexte d'apparition de DevOps?

Crise du logiciel et L'approche DevOps

- De 1965 à 1985: Crise du logiciel
- Approche waterfall
  - Projets de moindre qualité,
  - Délais non respectés,
  - Les budgets explosent,
  - Maintenance compliquée
  - Traçabilité mal gérée.
- À chaque fois qu'une nouvelle fonctionnalité est introduite, la régression s'accumule
- Mur de confusion entre le client (utilisateur) et l'équipe de développement

Crise du logiciel et L'approche DevOps

- Logiciel non envoyé en production,
- Investissement est nul.

De 1999 à 2001

- À partir de 1999 apparaît eXtreme Programming
  - Consiste à écrire les tests puis ensuite le code qui correspond aux tests
  - Ce qui permet d'enlever un mur de confusion entre les étapes de spésifications et le développement et permet une automatisation

Crise du logiciel et L'approche DevOps

- À partir de 2001 débute le mouvement Agile avec la division du projet en sprint.

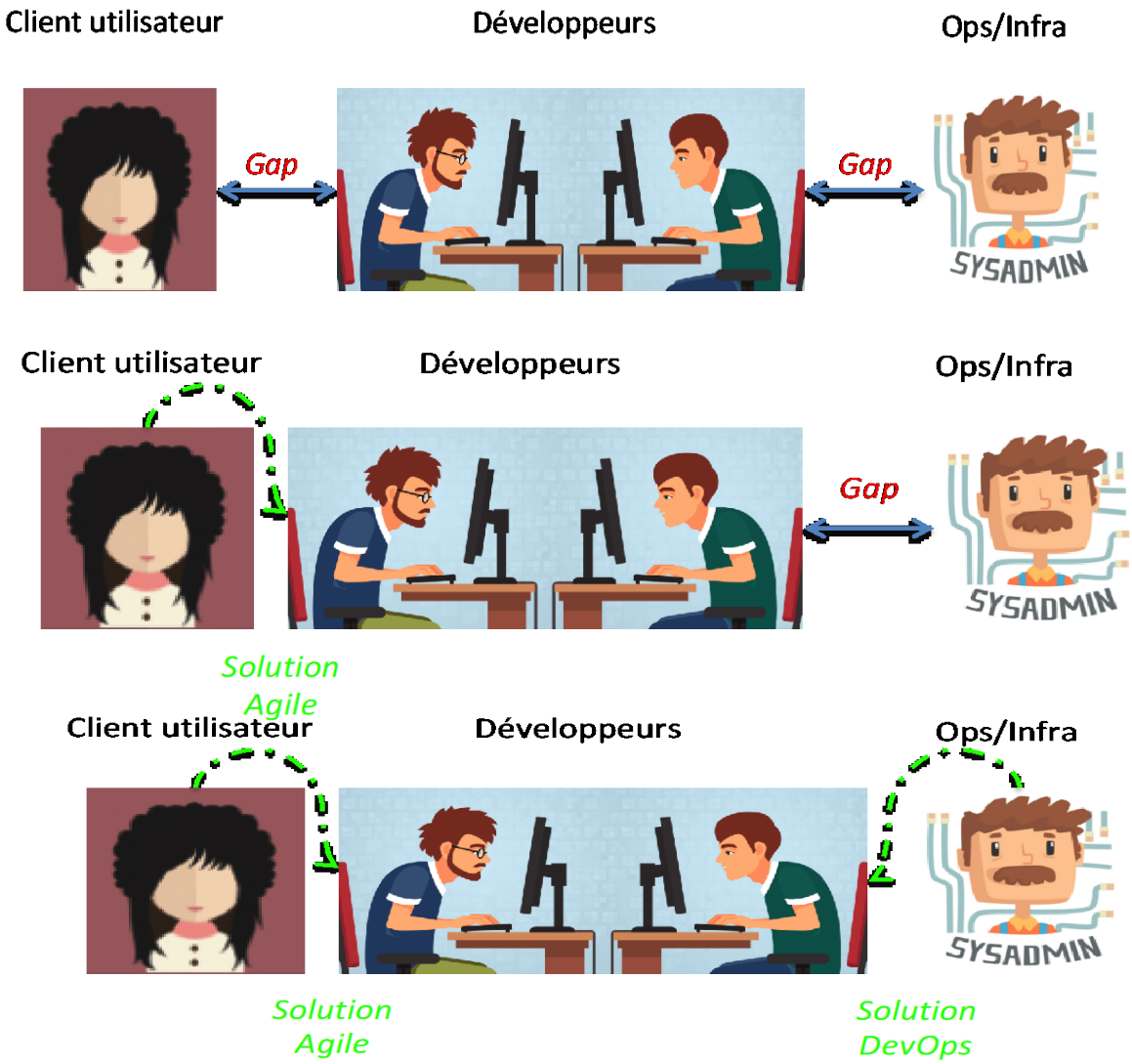
Compromis avec l'agilité:

- À la fin de chaque sprint, l'envoie du livrable directement de l'équipe de développement à l'équipe « opération »
- **Le mur de confusion entre utilisateurs et équipement de développement est résolu**
- Un **mur de confusion** persiste entre l'équipe de développement et l'équipe l'équipe d'infrastructure et d'opérations

L’approche DevOps

- En 2009, grâce au mouvement Devops accompagné de l’architecture en micro-services.
- Démarche visant la collaboration entre toutes les parties prenantes d'un projet informatique, depuis le développement jusqu'aux opérations
- Devops permet de bonnes performances de livraison logiciel.
- Voici les cinq critères permettant de mesurer la performance de livraison
  - le délais de réalisation
  - la fréquence de déploiement moyen par développeur
  - le délais moyen de remise en service lorsqu’une panne survient
  - le pourcentage de défaillance
  - la disponibilité du système

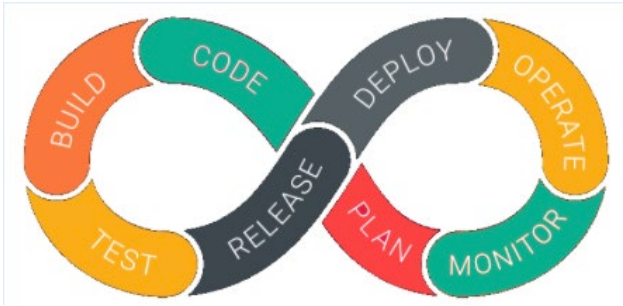
L'approche DevOps



3.6 – L'approche DevOps

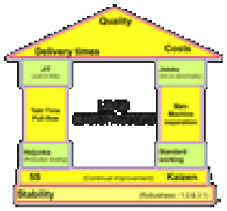
Agile	DevOps
Est intéressé par la livraison rapide et progressive de nouvelles fonctionnalités à valeur ajoutée	Est intéressé par les aspects de déploiement et d'opération du logiciel
Élimine l'écart entre les clients utilisateurs et l'équipe de développement pour accélérer les rétroactions concernant les exigences	Élimine l'écart entre l'équipe de développement et l'équipe d'infrastructure et opérations pour accélérer la transition vers la mise en production
Est centré sur les exigences fonctionnelles et non-fonctionnelles	Est centré sur la stabilité des opérations lors de mises en production répétées et s'assurer de l'état de préparation du logiciel prêt à être utilisé par ses utilisateurs
Met beaucoup d'accent sur la formation des membres d'équipe permettant, en cas de problème, que n'importe quel membre puisse aider à solutionner les problèmes	DevOps divise les tâches à des spécialistes mais s'assure d'une constante communication grâce à l'utilisation d'outils intégrés et à l'automatisation
Gère les livrables à l'aide de Sprints, soit la livraison de fonctionnalités à fréquence régulière (mois, 3 semaines, 2 semaines, …)	DevOps vise à consolider ses livrables selon la valeur pour la clientèle, dès que disponible

3.6 – L'approche DevOps

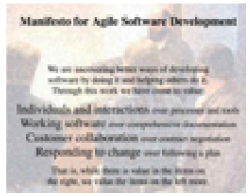




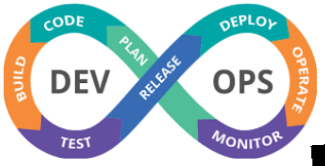
Toyota Kata



Approche Lean

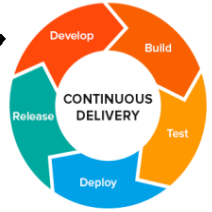


Manifeste Agile





Infrastructure Agile



Livraison Continue

Approche DevOps

- Une bonne performance de livraison logiciel induit
  - une bonne performance organisationnelle pour les organisations à but lucratif
  - et une bonne performance non commerciale pour les organisations à but lucratif.

Approche DevOps

Comment parvenir à une bonne performance de livraison?

3.6 – L'approche DevOps – Qu'est-ce que c'est?

5 principes clés:

- Qualité
- Petits lots
- Automatisation
- Amélioration continue
- Responsabilité

24 capacités
classées en 4 catégories:

- Techniques (11)
- Lean Management (4)
- Développement de produit Lean (4)
- Leadership transformationnel (5)

* Tiré de Nicole Forsgren PhD, Jez Humble et Gene Kim (2018). Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations, IT Revolution, ISBN: 978-1942788331.

3.6 – L'approche DevOps – Les 11 capacités TECHNIQUES

- *Automatisation des tests*
- *Automatisation du déploiement*
- *Développement sur le tronc*
- *Décalage à gauche de la sécurité*
- *Architecture découplée*
- *Équipes autonomes*
- *Intégration Continue*
- *Contrôle des versions*
- *Gestion des données de test*
- *Surveillance*
- *Notification proactive*

* Tiré de Nicole Forsgren PhD, Jez Humble et Gene Kim (2018). Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations, IT Revolution, ISBN: 978-1942788331.

3.6 – L’approche DevOps – Les 4 capacités du Lean Management



Limite des travaux en cours



Gestion visuelle



Rétroaction de la production



Approbation simple des modifications

* Tiré de Nicole Forsgren PhD, Jez Humble et Gene Kim (2018). Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations, IT Revolution, ISBN: 978-1942788331.

3.6 – L'approche DevOps – Les 4 capacités du Développement de produit Lean



Travailler en petits lots



Rendre le flux de travail visible



Recueillir et appliquer la rétroaction des clients



Expérimentation de l'équipe

* Tiré de Nicole Forsgren PhD, Jez Humble et Gene Kim (2018). Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations, IT Revolution, ISBN: 978-1942788331.

3.6 – L'approche DevOps – Les 5 capacités du Leadership Transformationnel

 *Vision*

 *Communication inspirante*

 *Stimulation intellectuelle*

 *Meneur supporteur*

 *Reconnaissance personnelle*

* Tiré de Nicole Forsgren PhD, Jez Humble et Gene Kim (2018). Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations, IT Revolution, ISBN: 978-1942788331.

DevOps

- DevOps

Capacités techniques	Lean Management
• Automatisation des tests • Automatisation du déploiement • Développement sur le tronc (code très proche de la production) • Décalage à gauche de la sécurité • Architecture découplée • Equipes autonomes • Intégration continue • Contrôle des versions • Gestion des données de test • Surveillance (log, ressource, serveur…) • Notification proactive (si quelque chose ne fonctionne pas il faut en être notifié avant que le client ne s'en rende compte)	• Limite des travaux en cours (Kanban l'utilise, il faut réduire le gaspillage en finissant ce qu'on commence) • Gestion visuelle (Kanban tableau) • Rétroaction de la production (feedback alimenté par la surveillance) • Approbation simple des modifications (un PO proche de l'équipe de développement)
Développement de produit Lean	**Leadership transformationnel**
• Travailler en petits lots (cela accélère le déploiement, on a plus souvent du feedback, on est en amélioration continue) • Rendre le flux de travail visible • Recueillir et appliquer la rétroaction des clients (le feedback du client améliore le produit) • Expérimentation de l'équipe (intégrer un sprint d'innovation pour tester de nouvelles techniques et technos sur le produit et la clientèle)	• Vision (convaincre le reste des équipes) • Communication inspirante (donner envie à l'équipe) • Stimulation intellectuelle (demander aux développeurs s'ils ont d'autres idées, cela permet de les garder motivé et donc productif) • Meneur supporter (le gestionnaire sait demander aux salariés ce qu'il peut faire pour les aider et de quoi ils ont besoin) • Reconnaissance personnelle (le gestionnaire reconnaît publiquement le succès et les efforts de l'équipe, il ne s'attribue pas le mérite)

3.6 – L'approche DevOps – Mesurer la « performance de livraison de logiciel »

- **5 « mesures clés »:**
  - Délai de réalisation → c'est le « temps de cycle » moyen
  - Fréquence de déploiement → en moyenne par développeur, pour pouvoir comparer les organisations entre elles, ou constater si on s'est amélioré malgré la variation du nombre de développeurs
  - Délai moyen de remise en service → suite à une panne
  - Pourcentage de défaillances → sur 100 changements, combien ont dû faire l'objet de corrections ou d'ajustements
  - Disponibilité → le système doit demeurer disponible
- **Pourquoi est-ce important?**
  - Parce que la performance financière (ou non commerciale) de l'organisation est directement liée à sa performance de livraison de logiciel, et ce même si le développement logiciel n'est pas sa mission première (ex. une Banque, une cie d'assurance, une cie de transport, etc.)

* Inspiré de Nicole Forsgren PhD, Jez Humble et Gene Kim (2018). Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations, IT Revolution, ISBN: 978-1942788331.

En somme

Différentes méthodologies:

- **SCRUM**, itérations à durée fixe qui reviennent
- **Kanban**, board où les items passent d'une colonne à l'autre en fonction de leur avancée
- **SAFe**, gérer de grands projets (plusieurs équipes en parallèle en mode Agile/Kanban/DevOps
- **DevOps**, ajout d'un rôle Ops dans l'équipe Dev Agile → Le DevOps, c'est de l'Agile++

Cas d’études

Différence entre DevOps et Scrum

- DevOps applique les principes Agile en dehors de l'équipe de développement.
- Scrum applique les principes Agile pendant le développement
- Scrum : Comment organiser l’équipe pour développer le logiciel ?
- DevOps : Comment collaborer, automatiser, livrer et exploiter le logiciel de manière continue ?

Cas d'études

Donnez quelques anti-patrons culturels qui nuisent à Devops?

Bonnes pratiques et pièges à éviter avec DevOps

2. Ne pas rechercher uniquement la vitesse

Multiplier les déploiements ne signifie pas être plus performant : qualité et stabilité restent essentielles.

3. Mesurer pour améliorer

Utiliser des indicateurs clés pour suivre la performance du développement et de la livraison.

4. Aller vite, sans sacrifier la qualité

DevOps vise une livraison rapide et fiable, et non la vitesse à tout prix.

5. Exploiter le feedback de production

Les retours réels permettent de détecter et corriger rapidement les problèmes et de réduire le travail à refaire.

DevOps = livrer rapidement + maintenir la qualité + apprendre du feedback.